

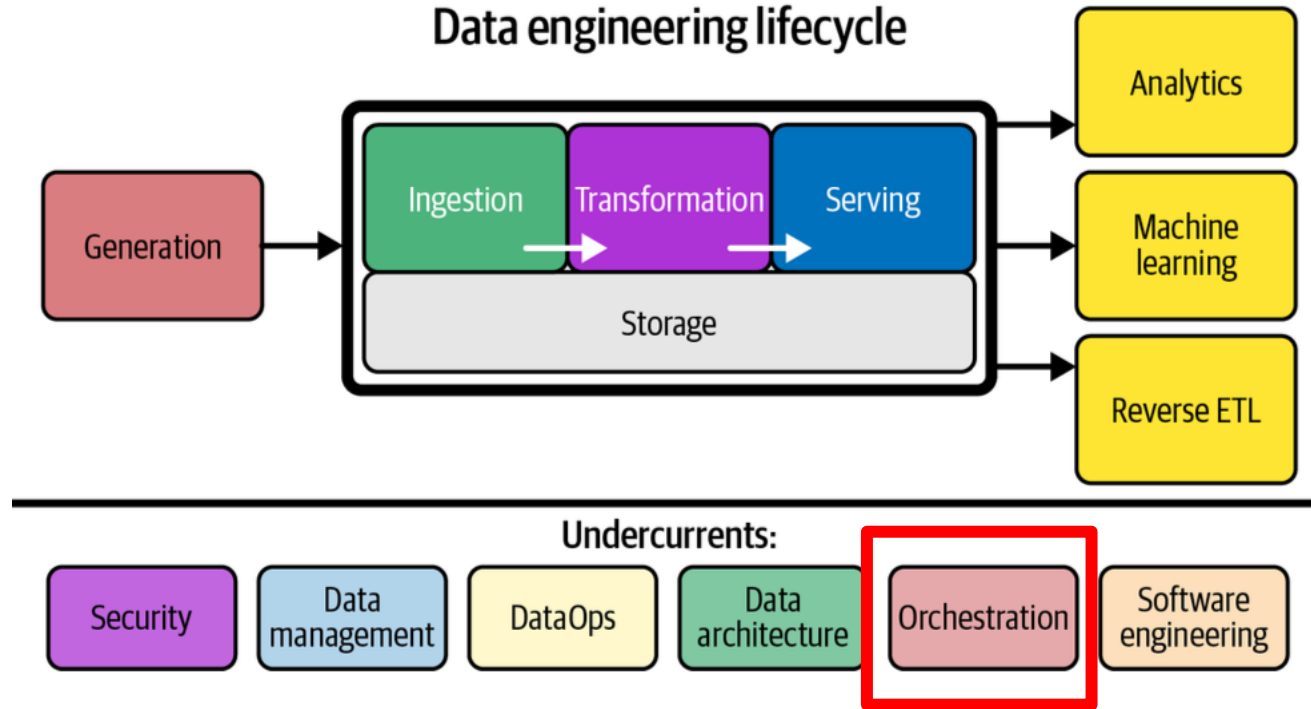
Data Engineering

Workflows Orchestration

UA.DETI.ED - 2025/26

José Luís Oliveira

The Data Engineering Lifecycle

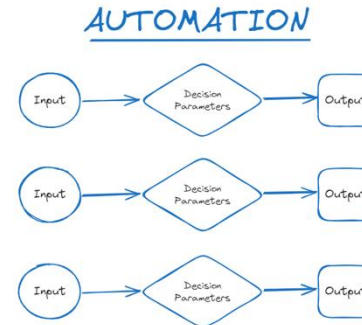


Introduction to Workflow Orchestration

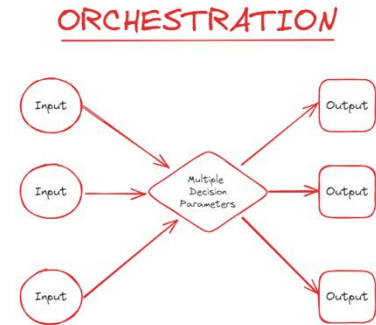
- ❖ Workflow orchestration serves as the critical coordinator for the entire data engineering lifecycle.
 - It ensures that complex tasks across ingestion, transformation, and serving execute in the proper sequence.
- ❖ Orchestration transforms a collection of isolated, disconnected scripts into a highly reliable and automated data platform.
 - It enables data teams to deliver data to consumers reliably at the promised time by systematically managing task execution.
- ❖ Modern orchestration tools, such as workflow orchestration platforms and software solutions, use technologies like artificial intelligence (AI), machine learning (ML) and low-code tools.

Automation versus Orchestration

- ❖ Automation is focused on executing a single, specific task without requiring manual human intervention.
 - An example of automation is a Python script designed to download a specific batch of NYC Taxi data every single day.
- ❖ Orchestration manages and coordinates multiple automated tasks to create a cohesive, end-to-end workflow.
 - It handles the overarching logic, ensuring the downstream transformation task only begins after the automated download is completely successful.



Automates single tasks,
to run on their own.

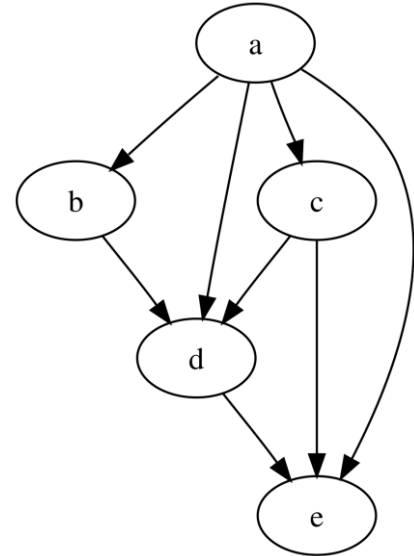


Automates many tasks,
to work as a process.

<https://kestra.io/blogs/2024-09-18-what-is-an-orchestrator>

Directed Acyclic Graphs (DAGs)

- ❖ Modern orchestration tools represent workflows as Directed Acyclic Graphs to clearly map out task dependencies.
 - The nodes in these graphs represent individual units of computational work, while the directed edges define the execution order.
- ❖ The "acyclic" property is a strict requirement to prevent infinite loops within the data pipeline.
 - Without this property, tasks could continuously trigger each other in a circle, leading to system deadlocks and wasted computational resources.



The Importance of Task Dependencies

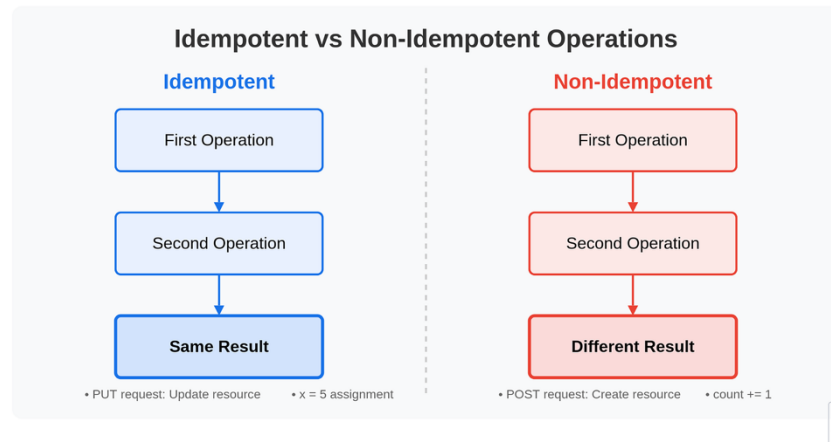
- ❖ Properly defined task dependencies maintain data integrity by enforcing a strict sequence of operations.
 - For instance, a data modeling task in the Gold layer must inherently wait for data cleansing to finish in the Silver layer.
- ❖ Dependencies prevent downstream processes from consuming incomplete, unvalidated, or corrupted datasets.
 - They act as logical safety gates that pause the entire pipeline if an upstream requirement fails to meet its predefined quality standards.

Scheduled vs. Event-Driven Triggers

- ❖ Orchestrators can trigger data workflows based on predefined time intervals using cron-like scheduling mechanisms.
 - This approach is ideal for batch processing workflows, such as updating daily financial reports or processing historical monthly records.
 - Time-driven, it runs even if data is not available, simple and predictable.
 - Example:
 - Run the pipeline every day at 2 AM to load data from the API to local storage.
- ❖ Alternatively, workflows can be triggered dynamically by specific external events or changing system states.
 - Event-driven triggers might successfully launch a pipeline the exact moment new data is generated.
 - Runs only when data arrives, more efficient than schedule trigger
 - Example:
 - Run the pipeline only when a new CSV file is uploaded to the input folder.

Pipeline Idempotency

- ❖ Idempotency is a foundational engineering practice
 - It ensures that running a pipeline multiple times yields the same final state.
 - This guarantees that rerunning a failed ingestion script does not result in duplicate records clogging up the data warehouse.
- ❖ Building idempotent workflows simplifies error recovery and overall system maintenance for the data engineering team.
- ❖ Engineers can confidently restart a DAG from the point of failure without needing to manually clean up partially processed data.



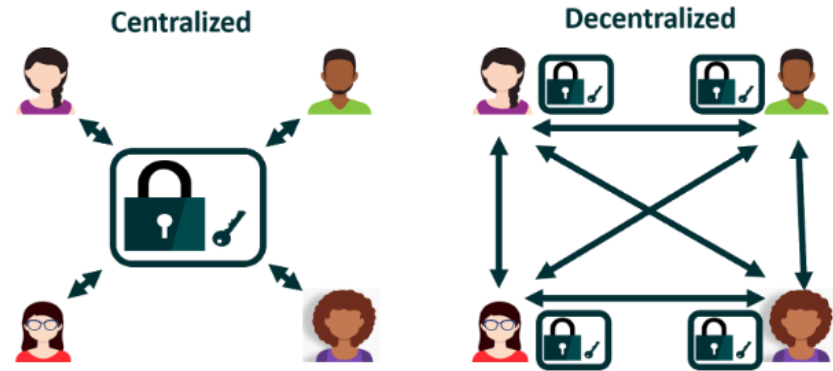
<https://medium.com/@jog.chinmay/understanding-idempotency-the-secret-to-reliable-data-processing-7a5dd5dde97d>

Centralised Orchestration Management

- ❖ Centralised orchestration provides a single, unified platform where an organisation manages all of its enterprise data pipelines.
- ❖ This approach allows a dedicated DataOps team to maintain high operational standards and enforce strict code gatekeeping rules.
- ❖ While centralisation greatly improves overall visibility, it can also create architectural bottlenecks and systemic risks.
 - A single poorly written DAG in a centralised system could potentially consume all available resources and halt the entire organisation's data flow.

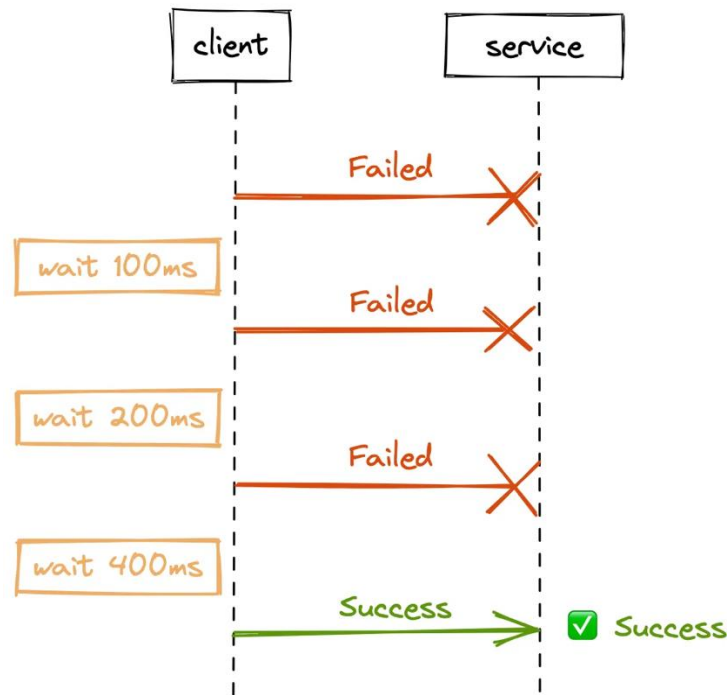
Decentralised Orchestration and Data Mesh

- ❖ Decentralised orchestration empowers individual domain teams to autonomously build, manage, and schedule their own data workflows.
- ❖ It is closely aligned with the Data Mesh paradigm, where domain teams act as the absolute owners of their specific data products.
- ❖ This approach drastically improves team agility but inherently increases the complexity of cross-team coordination.
 - Teams must establish robust messaging systems or API triggers to safely pass data dependencies between their different decentralised orchestrators.



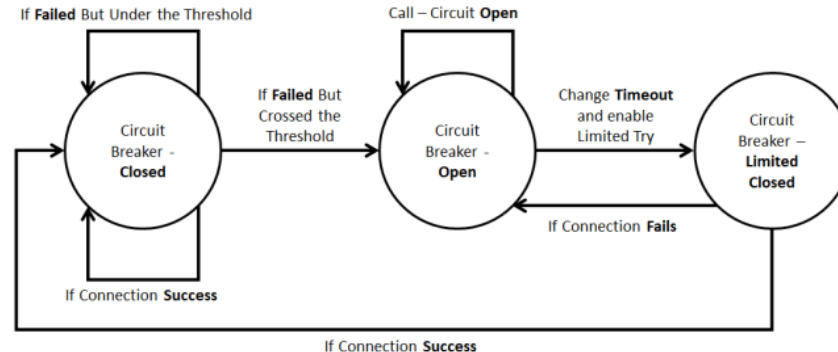
Handling Pipeline Failures

- ❖ Distributed data systems will inevitably experience failures, making resilient error handling an absolute necessity.
 - Orchestrators successfully mitigate these intermittent issues by automatically retrying failed tasks based on highly customisable retry limits.
- ❖ Implementing exponential backoff during retries is critical when interacting with fragile external public APIs.
 - This strategy prevents the orchestrator from overwhelmingly spamming an unresponsive API, giving the external service adequate time to recover.



Circuit Breaker Patterns

- ❖ The circuit breaker pattern is an advanced failure-handling mechanism designed specifically to protect broader system stability.
 - If a downstream service repeatedly fails, the orchestrator "opens the circuit" to halt all further execution attempts immediately.
- ❖ This highly effective pattern prevents cascading failures from spreading across the entire data engineering ecosystem.
 - Once the external service proves it is healthy again, the circuit closes, and the orchestrator safely resumes normal task execution.



Alerting and Observability

- ❖ Modern orchestration platforms are deeply integrated with alerting systems to promptly notify engineers of critical infrastructure issues.
- ❖ Automated notifications via email or team messaging apps ensure that unexpected data downtime is addressed as quickly as possible.
- ❖ Effective pipeline observability relies on the continuous and comprehensive logging of vital execution metadata.
- ❖ Capturing task run times, data volumes processed, and granular error logs enables DataOps teams to proactively optimise slow or failing workflows.

Observability

Understand what's happening inside a system and predict how it will behave in the future

Monitoring

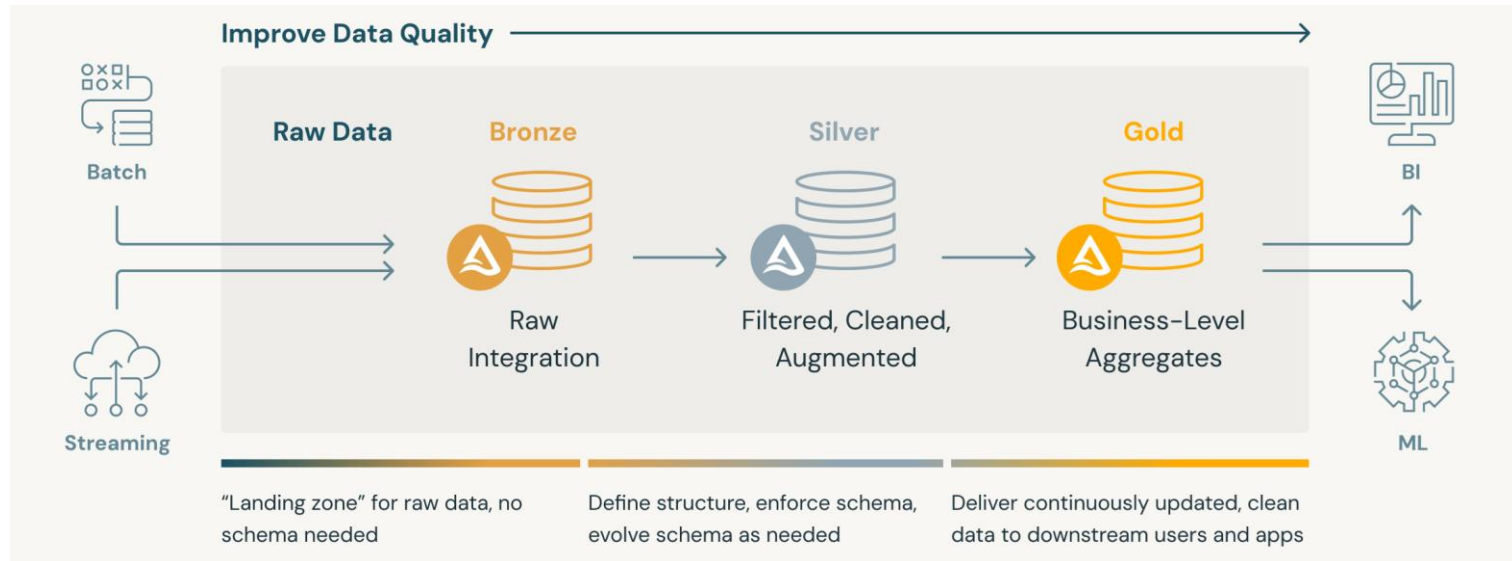
Continuous observation of a system to detect and alert on abnormal behavior

Telemetry

Automate collection and transmission of data from remote sources

Orchestrating the Medallion Architecture

- ❖ Workflow orchestration triggers the necessary transformations to 1) ingest data into the Bronze layer, 2) move raw, immutable data from the Bronze layer into the cleaned and standardised Silver layer; 3) manage the complex aggregations and joins strictly required for the Gold layer.



Orchestrating Machine Learning and Reverse ETL

- ❖ **Orchestration** extends far beyond traditional analytics to systematically manage the lifecycle of machine learning models.
- ❖ It seamlessly coordinates the extraction of features, the execution of model training scripts, and the deployment of predictive outputs to production.
- ❖ **Reverse ETL** workflows heavily rely on orchestration to push analytical insights back into active operational systems.
- ❖ An orchestrator might trigger a job that calculates lead scores in the data warehouse and then immediately syncs those scores directly into a CRM platform.

Apache Airflow

Introduction to Apache Airflow



- ❖ Apache Airflow is an open-source orchestrator used to build and coordinate complex data pipelines in a structured manner.
- ❖ It functions as a conductor that connects various systems to ensure data operations result in high-quality outputs.
 - Airflow can manage a wide variety of operations including loading data, transforming queries, and training machine learning models.
 - It is designed to handle the modern "data deluge" where volumes have increased from megabytes per day to gigabytes per minute.

Graph-Based Pipelines

- ❖ Pipelines in Airflow are represented as Directed Acyclic Graphs, which provide an intuitive visual overview of dependencies.
 - The acyclic property is essential because it prevents circular dependencies that would otherwise lead to execution deadlocks.
- ❖ Using a graph representation allows for parallelism by identifying tasks that are independent and can run at the same time.
- ❖ This approach is more efficient than monolithic scripts because it allows for granular failure handling and resource management.
 - If a single task fails in a graph, Airflow only needs to rerun that specific task and its downstream dependencies rather than the whole process.
 - Separating pipelines into small incremental tasks improves maintainability and makes the logic easier to test and debug.

Core Components of the Airflow Architecture

- ❖ Airflow is organised into five main components that work together to parse, schedule, and execute workflows.
 - **Scheduler**
 - The heart of Airflow, responsible for triggering scheduled workflows and submitting tasks to the executor when their dependencies are complete.
 - **Executor**
 - Determines how tasks are executed. Options include LocalExecutor (runs tasks on the same machine), CeleryExecutor (distributes tasks across worker nodes), and KubernetesExecutor (runs tasks as Kubernetes pods).
 - **Webserver**
 - Provides a user-friendly UI for monitoring, triggering, and debugging workflows. It visualises the DAGs, displays task status, and provides access to logs.
 - **Workers**
 - Execute the tasks assigned by the executor. In distributed setups with CeleryExecutor, workers run on separate machines to process tasks in parallel.
 - **Metadata Database**
 - Stores information about the state of tasks and DAGs. Typically uses PostgreSQL, MySQL, or SQLite to maintain execution history and status.

Key concepts in Airflow DAG

- ❖ **Operators** are the building blocks of Airflow that have a single responsibility to perform one specific piece of work.
 - Generic operators include the BashOperator for shell commands and the PythonOperator for executing arbitrary Python functions.
 - Specialised operators such as the EmailOperator or HTTPOperator are available to interact with specific external services or protocols.
- ❖ **Tasks** act as managers or wrappers around operators to handle execution details like errors and scheduling.
- ❖ **Task dependencies** define the execution order
 - Using the >> and << operators or the set_upstream() and set_downstream() methods
- ❖ **Sensors**: Special operators that wait for certain conditions to be true, such as file existence or table updates.
- ❖ **XComs**: Allow tasks to exchange small amounts of data with each other.

Defining Workflows Using Python Code

- ❖ Airflow allows users to define their pipelines as Python code, offering flexibility for building arbitrarily complex pipelines.
- ❖ Using code allows for dynamic generation of tasks or even entire DAGs based on external metadata or configuration files.
- ❖ Python is the primary language for Airflow, though Airflow 3 has introduced experimental support for other languages like Golang.
- ❖ Common methods for defining DAGs include the explicit definition, the context manager approach using the "with" keyword, and the Taskflow API.

Managing Task Dependencies

- ❖ Task dependencies define the execution order between various tasks.
- ❖ Airflow uses bitwise operators as a readable syntax to define dependencies
 - Using the `>>` and `<<` operators or the `set_upstream()` and `set_downstream()` methods
 - The right-shift operator (`>>`) is used to indicate that one task depends on the completion of the task to its left.
 - When a task fails, downstream tasks are marked as "upstream_failed" and will not execute, preventing the processing of invalid data.
- ❖ Dependency management ensures that consecutive tasks only start running once their upstream requirements are successfully met.

Scheduling Semantics and Intervals

- ❖ Airflow allows DAGs to be scheduled on regular time-based intervals such as hourly, daily, or weekly cycles.
- ❖ The scheduling logic is sophisticated enough to support cron-like expressions for complex or irregular timing requirements.
- ❖ Airflow divides time into discrete intervals, allowing pipelines to process only the "delta" or data that changed since the last run.
- ❖ This incremental approach provides cost and time benefits by eliminating the need to reprocess entire datasets every time.

Monitoring and Handling Failures

- ❖ The Airflow web interface provides extensive tools for inspecting the state of workflows and diagnosing potential issues.
- ❖ Built-in mechanisms allow for automatic retries of failed tasks, which helps pipelines recover from intermittent network or service errors.
 - Users can use the "Grid View" to see a historical record of how a DAG has performed over time and identify recurring bottlenecks.
 - If manual intervention is needed, the "Clear" function allows an engineer to reset a task state so that Airflow reruns it from the point of failure.

Backfilling and Versioning

- ❖ Backfilling is a powerful feature that allows users to run a DAG for schedule intervals that occurred in the past.
- ❖ Airflow 3 has introduced DAG versioning to track changes in code structure and dependency shifts directly in the database.
- ❖ Backfilling makes it easy to compute new datasets using historical data without manually triggering individual previous runs.
- ❖ DAG versioning lays the foundation for improved observability, allowing users to inspect historical structures through the UI and API.

Further Reading & Resources

- ❖ Data Pipelines with Apache Airflow, 2nd Edition, Chapters 1-7
 - <https://www.oreilly.com/library/view/data-pipelines-with/9781633436374/>
- ❖ Other workflow management sources

